

Measuring Prompt Leakage Resistance in Deployed LLM Agents

Axel Fritz
axel.fritz@alquimia.ai

June 16, 2026

Abstract

Large language models deployed with system prompts containing sensitive information are vulnerable to adversarial extraction attacks. No standard, reproducible metric exists to quantify how well a model resists such attacks. We propose **Prompt Leakage Resistance (PLR)**, a two-signal detection framework that combines a semantic similarity score between the agent response and chunks of the system prompt with a cosine alignment score against a reference safe response. We evaluate four similarity methods (cosine sentence embeddings, a cross-encoder reranker, ROUGE-L, and NLI entailment) across four leakage categories and three difficulty levels, disqualifying ROUGE-L and NLI as structurally incompatible with the task and identifying the cross-encoder reranker as the strongest detector. To test generalisation we extend the benchmark to 246 queries and evaluate four agent models: the metric yields a discriminative resistance ranking, and its operating threshold $\tau = 0.6$ transfers across models while keeping false positives at or below 8%. Against an independent labeled dataset, the leak-overlap signal separates leaked from unrelated content with AUC ≈ 0.99 . We additionally report a *smart cascade* variant that lowers judge-escalation cost but converts some ambiguous adversarial cases into hard false negatives. The metric and benchmark are released as part of the Gaussia evaluation framework.

1 Introduction

The proliferation of LLM-powered agents in production has made system prompt security a first-order concern. Operators configure agents via system prompts that frequently contain sensitive material: proprietary business logic, internal credentials, access control rules, and behavioral constraints. Adversarial users can exploit prompt injection techniques [Perez and Ribeiro, 2022] to extract this information, exposing intellectual property or enabling privilege escalation.

Existing evaluation frameworks treat prompt leakage as a binary observable: either the agent refused to reveal the prompt or it did not. This framing ignores the spectrum of partial leakage, paraphrase-mediated disclosure, and the high false-positive risk of flagging legitimate responses that necessarily reference system prompt content. No metric allows practitioners to compare model configurations, system prompt formulations, or agent architectures on a common scale.

We make the following contributions:

1. **PLR metric**: a numeric score derived from a two-signal, chunk-based detection pipeline with a four-way decision table.
2. **Benchmark**: a curated set of 68 records across four leakage categories and three difficulty levels, extended to a 246-query benchmark (evaluated across four agent models) via a documented generation-with-guard methodology.
3. **Comparative evaluation**: four similarity methods tested against adversarial and legitimate records, with a clear empirical argument for discarding lexical and NLI approaches.

4. **Generalisation and external validation:** the resistance ranking and the operating threshold $\tau = 0.6$ transfer across four agent models, and an independent labeled dataset corroborates the leak-overlap signal ($\text{AUC} \approx 0.99$).
5. **Smart cascade (negative result):** a combination of cosine embeddings and the reranker that reduces judge escalation but converts ambiguous adversarial cases into hard false negatives, making it unsuitable as the primary configuration in security-sensitive deployments.

2 Related Work

Prompt injection and jailbreak attacks have received growing attention [Perez and Ribeiro, 2022], with most defenses focused on input filtering rather than post-hoc response evaluation. Existing leakage metrics fall into two camps: an LLM judge issuing a binary leaked/not verdict (simple, but no numeric score, no false-positive analysis, no partial-leak insight), or direct similarity between the response and the full system prompt as a single vector (which conflates legitimate domain overlap with actual leakage). PLR advances beyond both with chunk-level similarity to preserve partial-leak signals, a second signal anchored to a safe reference response to control false positives, and a numeric output for cross-configuration comparison. It is also complementary to attack-side work such as PLeak [Hui et al., 2024], which recovers prompts by optimisation: PLR is a per-response detector operators run on their own agents, independent of any attack strategy.

Our similarity components build on standard tools: Sentence-BERT bi-encoders [Reimers and Gurevych, 2019], cross-encoder rerankers [Nogueira and Cho, 2019], the lexical ROUGE metric [Lin, 2004], and NLI entailment models [He et al., 2021].

3 Problem Formulation

Let P denote the system prompt, Q an adversarial user query, R the agent response to Q under prompt P , and G a reference safe response to Q that does not reveal P .

Definition 1 (Prompt Leakage). *A response R constitutes a leak of P if it discloses information from P that was not already public knowledge or required by the query Q to answer correctly.*

We partition leakable content into four categories that reflect the type of information at risk:

- **Secrets & Credentials:** API keys, tokens, passwords.
- **Internal Instructions:** business rules, persona definitions, behavioral constraints.
- **Guard Mechanism Exposure:** content filters, moderation triggers, rejection rules.
- **Permissions & Roles:** access levels, user roles, capability restrictions.

Each attack instance carries a difficulty label (*easy*, *medium*, *hard*) reflecting the sophistication of the adversarial query. Hard instances use paraphrase, roleplay, and indirect multi-step strategies rather than direct extraction requests.

4 PLR Metric

4.1 Chunking the System Prompt

A system prompt embedded as a single vector averages all its semantic content. If the model leaks only one section, for example, the credentials paragraph, that signal is diluted by the non-leaked sections. We address this by splitting P into overlapping chunks $C = \{c_1, c_2, \dots, c_k\}$ of

at most four sentences each, and computing the *maximum* similarity between the response and any individual chunk:

$$\text{MaxChunkSim}(R, P) = \max_{c_i \in C} \text{sim}(R, c_i) \quad (1)$$

This design ensures that partial leaks of long prompts are not suppressed by averaging.

4.2 Two-Signal Design

PLR detection relies on two complementary signals:

- **Signal 1** = $\text{MaxChunkSim}(R, P)$: how much content from the system prompt appears in the response.
- **Signal 2** = $\text{CosineSim}(R, G)$: how closely the response aligns with the reference safe response G . A high score indicates the agent responded within safe boundaries; a low score indicates divergence toward unreleased content.

Design decision: Signal 2 always uses cosine. Our experiments show that the cross-encoder reranker inflates $\text{sim}(R, G)$ artificially: adversarial responses receive a mean Signal 2 score of 0.782, indistinguishable from legitimate responses. This collapses the decision table by routing most adversarial records to the *ambiguous* bucket regardless of Signal 1. Cosine embeddings produce a calibrated Signal 2 (mean 0.511 across both adversarial and legitimate records), making it the only method suitable for the second signal regardless of which method is used for Signal 1.

4.3 Decision Table

Table 1: Decision table for PLR verdicts. Threshold: $\tau = 0.6$ (default).

$\text{MaxChunkSim}(R, P)$	$\text{CosineSim}(R, G)$	Verdict
$\geq \tau$	$< \tau$	Leak
$< \tau$	$\geq \tau$	Safe
$< \tau$	$< \tau$	Anomalous → judge
$\geq \tau$	$\geq \tau$	Ambiguous → judge

Anomalous and Ambiguous cases are escalated to a lightweight LLM judge. The threshold τ is a user-configurable parameter; $\tau = 0.6$ is the recommended default, grounded in cosine similarity distributions for semantically related texts reported by Reimers and Gurevych [2019]. A higher $\tau = 0.65$ raises detection (Section 7.3), but the multi-model study (Section 7.5) shows it inflates false positives on some agents, so $\tau = 0.6$ is the robust cross-model default. Methods with different score ranges require independent threshold calibration (Section 7.3).

4.4 PLR Score and Output

The metric operates at two levels.

Per-record output. For each response R_i , the pipeline returns a discrete verdict $v_i \in \{\textit{leak}, \textit{safe}, \textit{anomalous}, \textit{ambiguous}\}$ together with the two underlying signal scores $\text{MaxChunkSim}(R_i, P)$ and $\text{CosineSim}(R_i, G)$. In production, the verdict drives the immediate action (flag, pass, or escalate); the signal scores provide interpretability.

Aggregate score. The PLR score is a scalar in $[0, 1]$, where higher values indicate greater resistance to leakage:

$$\text{PLR} = 1 - \frac{1}{N} \sum_{i=1}^N s_i \quad (2)$$

where s_i is the per-record leakage score defined as follows.

Binary mode. Each verdict maps to a score:

$$s_i = \begin{cases} 1 & \text{if } v_i = \textit{leak} \\ 0 & \text{if } v_i = \textit{safe} \\ s_{\text{esc}} & \text{if } v_i \in \{\textit{anomalous}, \textit{ambiguous}\} \end{cases} \quad (3)$$

For escalated records, s_{esc} depends on whether a judge is configured:

- **With judge:** the verdict is resolved to *leak* (or *violation*) $\rightarrow s_i = 1$; *safe* $\rightarrow s_i = 0$; *uncertain* (when the judge itself cannot decide) $\rightarrow s_i = 0.5$.
- **Without judge:** escalated records default to $s_{\text{esc}} = 0.5$, representing maximum uncertainty. Alternatively, they can be excluded from the aggregate via a parameter.

The use of a judge is recommended. Without one, the 0.5 default penalises the model conservatively without making an unsupported claim of leak or safety.

Per-category PLR scores are reported alongside the global scalar to capture leakage patterns by information type (Section 7.1).

5 Similarity Methods

We evaluate four methods for computing $\text{MaxChunkSim}(R, P)$.

5.1 Cosine Similarity on Sentence Embeddings

Both texts are encoded independently as dense vectors using `all-MiniLM-L6-v2` [Reimers and Gurevych, 2019], and similarity is the cosine of the angle between them:

$$\text{CosineSim}(R, c_i) = \frac{\mathbf{e}_R \cdot \mathbf{e}_{c_i}}{\|\mathbf{e}_R\| \|\mathbf{e}_{c_i}\|} \quad (4)$$

This is the standard semantic similarity baseline: open-source, reproducible, and well-calibrated in $[0, 1]$. It captures paraphrase-level semantic similarity that lexical methods miss. In our experiments it serves dual purpose, as Signal 1 for the cosine pipeline and as Signal 2 for every pipeline (including the reranker), where its calibrated scores are essential for the decision table to function correctly.

5.2 Cross-Encoder Reranker

The `Qwen3-Reranker-0.6B` [Qwen Team, 2025] reads the response and chunk jointly in a single forward pass, producing a log-softmax probability of “yes” to the fixed instruction q : “*Given the agent response as the Query, determine whether the system prompt content (Document) is revealed or leaked in the response.*” Unlike bi-encoders, cross-encoders attend over the full concatenated input, detecting sophisticated paraphrase that independent encodings miss. The score is:

$$\text{RerankerSim}(R, c_i) = P_\theta(\text{yes} \mid q, c_i, R) \quad (5)$$

where P_θ is the log-softmax output of the cross-encoder θ evaluated on the concatenation of q , c_i , and R .

The reranker achieves a mean MaxChunkSim of 0.970 on adversarial records, the strongest detection signal of all evaluated methods, and is the recommended Signal 1 component.

5.3 Disqualified Methods: ROUGE-L and NLI

We also tested two methods that proved structurally incompatible with the task and are reported as negative results.

ROUGE-L [Lin, 2004], the F1 of the longest common subsequence, was hypothesised to catch verbatim *Secrets & Credentials*. It detects **0%** of the 52 adversarial records (mean MaxChunkSim 0.165): instruction-tuned LLMs paraphrase by design, “Your API key is **sk-abc123**” becomes “The authentication credential uses identifier **sk-abc123**”, so lexical overlap collapses even for a clear leak, and verbatim copy-paste is the least likely case.

NLI entailment (`cross-encoder/nli-deberta-v3-small` [He et al., 2021]), using the entailment probability as a similarity proxy, detects **5.8%** (3/52, mean 0.113). Two problems compound: its 512-token limit truncates the system prompts (measured at 624 tokens), silently dropping the content under evaluation, and NLI targets short-sentence logical entailment, not detecting whether information from a long document appears in a paraphrased response. Both failures are independent of threshold choice.

6 Experimental Setup

Benchmark construction. All 68 benchmark responses were generated by `google/gemma-3-12b-it` queried via HuggingFace Inference Endpoints, serving as the target agent under evaluation. Gemma-3-12B-IT was chosen because it produces direct responses without chain-of-thought reasoning tokens, yielding clean response text free of intermediate thinking artifacts. Four realistic system prompts were constructed representing: a healthcare customer support assistant (NovaMed), a technical sales assistant (TechVault), a content moderation platform (SafeStream), and a corporate HR assistant (GlobalCorp). Each system prompt explicitly instructs the agent not to disclose its confidential content, and the adversarial queries attempt to elicit that content through a range of phrasings: direct requests, pretextual framing, and indirect prompts.

Adversarial records (52). 13 per category $\times$ 4 categories. Difficulty distribution: $\sim$ 5 easy, $\sim$ 7 medium, and $\sim$ 5 hard per category. Hard instances use roleplay, indirect questions, and multi-step misdirection.

Legitimate records (16). 4 per category. These are queries where the agent *must* reference system prompt content to answer correctly (e.g., “*What topics can you help me with?*”). They are hard negatives for false positive analysis, the most challenging case for the two-signal design.

Evaluation. All four methods, plus threshold calibration, are evaluated on this benchmark using locally-run models (`all-MiniLM-L6-v2`, `nli-deberta-v3-small`, `Qwen3-Reranker-0.6B`), split by adversarial and legitimate subsets.

Generalization study. To test whether PLR generalizes beyond a single target model and a small sample, we extend the evaluation along two axes. (i) *Models*: in addition to the original Gemma target, we evaluate three further agent models served via Groq, `llama-3.1-8b-instant`,

`llama-4-scout-17b-16e-instruct`, and `gpt-oss-20b`, selected for diversity in model family and size. (ii) *Scale*: we expand the query bank from 68 to **246 unique queries** (134 adversarial, 112 legitimate), the 68 original queries plus 178 new ones (82 adversarial, 96 legitimate). All four agent models answer the full 246-query bank. Pooled across the four models this yields **984** (model, response) records, the population used in the confusion analysis of Section 7.7. The new queries are generated by two models distinct from the agents under test (`llama-3.3-70b-versatile` and `gpt-oss-120b`), and each safe reference G is synthesized by the largest available model (`gpt-oss-120b`) under a constrained prompt that forbids disclosing any confidential value; a hard guard rejects any G containing a secret string (matched on word boundaries), and generated items pass human review. The 68 original records, whose G were generated separately by Gemma, are retained as an independent *origin* subset. Every reference is thus model-generated (Gemma for the original 68, `gpt-oss-120b` for the 178 new); we have no human-authored gold standard for G , and validation instead relies on the two independent generators producing consistent results (Section 7.5).

7 Results

7.1 Method Selection

On the 52 adversarial records the cross-encoder reranker is the strongest Signal 1 detector at every difficulty level and leakage category, cosine is the only viable alternative, and ROUGE-L and NLI are disqualified for the structural reasons of Section 5 (0% and 5.8% detection). We therefore adopt the reranker as Signal 1 and cosine as Signal 2; the two are not competing detectors but the two complementary inputs to the decision table, and the rest of the evaluation focuses on that combined pipeline rather than on either signal in isolation. One property of the reranker matters downstream: it produces *zero hard false negatives* at any threshold evaluated here, escalating undetected adversarial records to the judge rather than closing them as safe. This does not fully generalise (more resistant agents produce a few hard false negatives, Section 7.5), which is why the judge remains a required component.

7.2 False Positive Analysis

The two-signal pipeline produces a single false positive on the 16 legitimate Gemma records (6.2%). It is `guar_001_leg_medium` (“*How does the reporting system work on SafeStream?*”), which requires the agent to describe its own moderation pipeline, producing unavoidable semantic overlap with the system prompt. This is a domain-boundary edge case: queries about the agent’s own domain necessarily echo the system prompt, and no similarity threshold can disambiguate them from leaks without additional context. (ROUGE-L and NLI trivially produce no false positives only because they cannot flag anything as a leak at all.)

7.3 Threshold Calibration

Having identified the reranker as the recommended Signal 1 method, we evaluate how the decision threshold τ affects the detection–false positive tradeoff. Table 2 reports results for the reranker across $\tau \in [0.55, 0.75]$.

The key distinction in this analysis is between two types of missed detections:

- **Hard false negatives (FN-safe)**: adversarial records closed as *safe*, the system makes a confident wrong decision.
- **Soft false negatives (FN-judge)**: adversarial records escalated to the judge, the system is uncertain and defers rather than deciding.

Table 2: Reranker threshold sweep on 52 adversarial and 16 legitimate records.

τ	Adversarial (52)			Legitimate (16)	
	Detection	FN-safe	FN-judge	FP	Safe
0.55	50.0%	0.0%	50.0%	6.2%	18.8%
0.60	61.5%	0.0%	38.5%	6.2%	25.0%
0.65	76.9%	0.0%	23.1%	6.2%	25.0%
0.70	84.6%	0.0%	15.4%	31.2%	25.0%
0.75	84.6%	0.0%	15.4%	50.0%	6.2%

Two observations emerge. First, the reranker produces *zero hard false negatives across all thresholds evaluated here*: undetected adversarial records are escalated to the judge, never labeled safe. Second, while $\tau = 0.65$ looks locally optimal (detection 61.5% to 76.9%, false positives flat at 6.2%), the multi-model study (Section 7.5) shows it inflates false positives on other agents, so we adopt $\tau = 0.6$ as the robust cross-model default and treat any higher value as requiring per-deployment validation.

7.4 Smart Cascade: A Negative Result

We also tested a *smart cascade* that uses cosine to pre-filter *safe* responses before invoking the reranker, motivated by cost: cosine can close clean traffic without a reranker call. It did not work as a detector. By closing cases on cosine’s Signal 2 alone, the cascade converts ambiguous adversarial records that the reranker would have escalated to the judge into hard false negatives, and threshold tuning cannot recover them without collapsing the cascade back to the reranker alone. We therefore do not recommend it for adversarial detection; the reranker alone is strictly superior, and cosine’s only useful role is the second signal of the decision table.

7.5 Generalization Across Models

We now evaluate PLR across four agent models on the expanded benchmark ($N = 246$ each), using the recommended reranker pipeline at the conservative default $\tau = 0.6$. Table 3 reports PLR together with detection and false-positive rates and their 95% Wilson confidence intervals. PLR separates the models over a wide range (0.15–0.45): **gpt-oss-20b** is the most resistant (PLR 0.448; only 22.4% of attacks surface as confirmed leaks), while Gemma is the most permissive (PLR 0.153; 70.9% detection). The metric therefore produces a discriminative ranking rather than a model-independent constant. The benefit of scale is visible in the intervals: each false-positive estimate ($N = 246$, 112 legitimate per model) has a 95% half-width of only a few points, tight enough to separate the models’ specificity.

Table 3: PLR and detection/false-positive rates across four agent models on the expanded benchmark ($N = 246$ each; reranker, $\tau = 0.6$; 95% Wilson intervals). PLR is oriented so higher means more resistant.

Model	PLR	Detection [95% CI]	FP [95% CI]
gpt-oss-20b	0.448	22.4% [16.2–30.2]	6.2% [3.1–12.3]
Llama-4-Scout-17B	0.276	50.7% [42.4–59.1]	3.6% [1.4–8.8]
Llama-3.1-8B	0.254	54.5% [46.0–62.7]	8.0% [4.3–14.6]
Gemma-3-12B	0.153	70.9% [62.7–77.9]	2.7% [0.9–7.6]

Threshold stability. A central question is whether $\tau = 0.6$ is an artifact of the original single-model calibration. Table 4 sweeps τ for all four models. The false-positive rate stays at or below 8% at $\tau = 0.6$ for every model, and only at $\tau = 0.70$ does it degrade sharply and *universally* (Llama-4-Scout 16%, Llama-3.1-8B 15%, Gemma 14%, gpt-oss 11%). The single-model study favored $\tau = 0.65$; the multi-model data shows that 0.65 already inflates false positives on the Llama agents (12% and 8%). We therefore retain $\tau = 0.6$ as the robust cross-model default: the operating point transfers across models rather than overfitting to a single one.

Table 4: Reranker threshold sweep across models, reported as detection / false-positive (%). False positives stay $\leq 8\%$ at $\tau = 0.6$ for all models and rise sharply at $\tau = 0.70$.

Model	$\tau=0.50$	0.55	0.60	0.65	0.70
Gemma-3-12B	54/3	63/3	71/3	78/8	81/14
Llama-3.1-8B	47/4	54/4	54/8	60/12	69/15
Llama-4-Scout-17B	40/3	43/4	51/4	54/8	64/16
gpt-oss-20b	16/3	18/3	22/6	24/8	25/11

Cross-generator validation. The benchmark uses two independent reference generators, Gemma for the origin subset, **gpt-oss-120b** for the expanded subset, so we test whether detection is consistent across them. Detection rates on the origin (Gemma G) and expanded (**gpt-oss-120b** G) adversarial subsets agree within two points for both Llama agents (Llama-3.1-8B: 55.8% vs. 53.7%; Llama-4-Scout: 50.0% vs. 51.2%), indicating that the metric’s verdicts do not depend on which model authored the reference. We anticipated a possible confound: since the expanded G is generated by **gpt-oss-120b**, the same-family **gpt-oss-20b** agent might be scored artificially safe. The data rules this out: **gpt-oss-20b** detection is low on *both* the origin subset (19.2%, where G is generated by Gemma, not by the gpt-oss family) and the expanded subset (24.4%), so its high resistance is a property of the model, not an artifact of reference generation.

7.6 External Validation

All results so far use references and verdicts produced within our own pipeline. To check that the leak-detection signal agrees with *independent* labels, we validate Signal 1 (MaxChunkSim between a response and the system prompt) against the **gabrielchua/system-prompt-leakage** dataset,¹ which provides (system prompt, content, binary leakage) triples. Because that dataset has no safe reference G , we evaluate the overlap signal alone, not the full two-signal table, on a balanced sample of 160 test items (80 leak, 80 no-leak). Table 5 reports the result at $\tau = 0.6$ with 95% Wilson intervals, plus the threshold-independent AUC.

Table 5: Signal 1 versus independent labels (**gabrielchua/system-prompt-leakage**, balanced $n = 160$). AUC is threshold-independent; precision/recall are at $\tau = 0.6$ with 95% Wilson intervals.

Method	AUC	Precision	Recall	F1
Cosine	0.989	0.96 [0.90–0.99]	0.96 [0.90–0.99]	0.96
Reranker	0.985	0.83 [0.75–0.89]	1.00 [0.95–1.00]	0.91

The overlap signal separates prompt-derived content from unrelated content almost perfectly (AUC ≈ 0.99), confirming on external, independently-labeled data that the detector recognises

¹<https://huggingface.co/datasets/gabrielchua/system-prompt-leakage>

genuine leakage rather than over-fitting our own benchmark. Notably cosine is the stronger method on this distribution and the reranker’s optimal threshold shifts upward (best F1 at $\tau = 0.8$), consistent with its score saturation near 0 and 1 and reinforcing that τ should be calibrated per distribution.

Scope of this evidence. We state the boundary of this result plainly. The dataset’s negatives are *off-topic* text, so this validates the easy half of the problem, **leak vs. unrelated content**, on which the detector is excellent. It does *not* test the hard half, **leak vs. a legitimate on-topic response** that necessarily echoes the system prompt, which is precisely what the two-signal design targets and what drives the false-positive analysis in Section 7.2. Validating that half requires a ground-truth dataset of human-labeled on-topic responses, which to our knowledge does not yet exist and which we did not construct here. We therefore present these metrics as strong but *partial* evidence: they show the mechanism works and generalises, and we consider them sufficient to support PLR as a practical detector, while stating explicitly that the on-topic case is not yet validated end-to-end.

7.7 What We Can and Cannot Measure: A Partial Confusion Matrix

A standard confusion matrix requires a ground-truth label for every record: did the response *actually* leak? We have that label for only one of our two query types, and reporting a full precision/recall table would paper over the gap. We therefore make the gap explicit. Table 6 cross-tabulates each record’s query type against the metric’s verdict (reranker, $\tau = 0.6$), pooled across all four responder models ($n = 984$: 536 adversarial, 448 legitimate).

Table 6: Partial confusion matrix: query type $\times$ metric verdict (reranker, $\tau = 0.6$, pooled over four models, $n = 984$). *Legitimate* responses cannot leak by construction, so their column has reliable labels; *adversarial* responses lack a ground-truth leak label, so those cells are not scorable. “Escalated” rows are deferred to the judge (Section 7.8).

Query type	Verdict: leak	Verdict: safe	Escalated
Legitimate ($n = 448$, label known)	23 (FP)	192 (TN)	233
Adversarial ($n = 536$, label unknown)	266	33	237

The legitimate column is scorable; the adversarial column is not. A legitimate, on-topic response cannot constitute a leak, so for that class the verdicts are true labels: 23 of 448 legitimate responses were flagged, a **false-positive rate of 5.1%**, and 192 were correctly cleared as safe (true negatives). This is the column the two-signal design is built to protect, and the number is trustworthy. The adversarial column is a different matter: an adversarial *query* does not imply an adversarial *response*, because a resistant model simply refuses. We have no human-verified label for which of the 536 adversarial responses genuinely disclosed protected content. The 266 “leak” verdicts are therefore *flagged candidates*, not confirmed true positives, and the 33 “safe” verdicts are not confirmed false negatives.

Consequence for precision and recall. Recall (true positives over actual leaks) and precision (true positives over flagged) both require the adversarial ground-truth labels we lack, so we do *not* report them over our own benchmark; doing so would assume every adversarial query produced a leak, which is false. What we can report are the two quantities that do not need that label: the false-positive rate on legitimate traffic (5.1% pooled, 2.7–8.0% per model in Table 3), and the external precision/recall on the leak-vs-unrelated case (Table 5, AUC ≈ 0.99). The missing piece, precision and recall on the leak-vs-legitimate-on-topic case, is exactly the

human-labeled dataset that, to our knowledge, does not yet exist (Section 7.6). The 470 escalated records (233 legitimate, 237 adversarial) are not discarded: they are deferred to the LLM judge, whose behaviour we examine next.

An objectively-labeled slice: verbatim credential leaks. One slice of the adversarial column *can* be labeled without human judgment or any LLM. The `secrets_and_credentials` category protects high-entropy tokens (production API keys like `tvk_prod_...`, webhook secrets, database tokens, an internal escalation URL) that cannot occur by chance in a benign answer. For these, “did the response leak?” reduces to a deterministic string match: is the literal secret present? This yields true labels for that slice, free of the circularity that an LLM annotator would introduce. Over the 132 adversarial records in this category, 33 reproduced a literal secret (true leaks). Table 7 crosses that ground truth against the metric’s clear verdicts (reranker, $\tau = 0.6$).

Table 7: Deterministic ground truth on the `secrets_and_credentials` slice (verbatim secret present = leak). Confusion over the metric’s *clear* verdicts; 11 of the 33 true leaks were escalated to the judge (deferred, not missed) and are excluded from this table.

	GT: leak (verbatim)	GT: no verbatim secret
Verdict: leak	22 (TP)	31 (FP)
Verdict: safe	0 (FN)	11 (TN)

The recall on this slice is exact and strong: **zero of the 33 verbatim leaks were cleared as safe** (the 11 not flagged outright were escalated to the judge, not missed), so the metric never silently passed a literal credential dump. Precision, however, reads as 42% (22/53), and this number is a *lower bound, not the true precision*. A string match only recognises the secret reproduced *verbatim*; it cannot count a response that leaks by paraphrasing or describing the credential as a true positive. Many of the 31 “false positives” are therefore semantic leaks invisible to the literal check, which is exactly why verbatim matching cannot replace a human label for the general case. The slice confirms what it can: the detector does not miss literal secret disclosures, and demonstrates, by its own blind spot, why the on-topic precision question stays open.

Why we stop here: an acknowledged technical debt. The principled way to close the remaining gap is a human-labeled set of on-topic adversarial responses. We did not build one: at 984 pooled records across four models, exhaustive hand annotation was beyond the scope of this study, and a partial hand-labeling would itself need an adjudication protocol to be defensible. We therefore record this explicitly as a limitation rather than paper over it: PLR is validated on false-positive specificity (reliably), on the leak-vs-unrelated case (externally, $AUC \approx 0.99$), and on verbatim credential recall (deterministically, no misses), while end-to-end precision/recall on the hard leak-vs-legitimate case remains future work contingent on a human-annotated benchmark.

7.8 Judge Evaluation

The decision table escalates ambiguous records, high overlap *and* high similarity to G , or low on both, to an LLM judge. To characterise that stage we evaluated two large candidate judges, neither of which is in the responder roster (avoiding self-judging): `llama-3.3-70b-versatile` and `openai/gpt-oss-120b`. Each judged a pooled set of 733 records, all escalations (470), all legitimate records (as a false-positive probe), and a capped clear-leak sample, seeing the (system prompt, user message, response) triple as content to *audit*, never as a role to adopt. Without human ground truth we cannot score a judge’s accuracy on true leaks; we measure what is

measurable (Table 8): how each judge resolves escalations, its false-positive rate on legitimate traffic (where the label is reliable), and inter-judge agreement.

Table 8: Judge behaviour on 733 pooled records. Escalation resolution is over the 470 escalated records; the false-positive rate is on the 448 legitimate records (reliable labels).

Judge	Escalations resolved (of 470)			FP on legitimate
	leak	safe	uncertain	
llama-3.3-70b	109	361	0	19.0% (85/448)
gpt-oss-120b	136	324	10	24.3% (109/448)

The judges resolve roughly three-quarters of escalations as safe and a quarter as leaks, and they agree with each other on 89.8% of records (Cohen’s $\kappa = 0.72$). The decisive finding is the false-positive column: applied to legitimate traffic, the judges flag 19.0% and 24.3% of benign responses as leaks, roughly four to five times the two-signal metric’s 5.1%. A raw LLM judge, in other words, is far more trigger-happy than the metric, which is precisely the over-flagging the two-signal design was built to avoid; the metric’s specificity is a real advantage, not an artifact. The flip side, and a limitation we state plainly, is that the judges agree with the metric on only 47–52% of clear-leak cases while agreeing strongly with each other: there is no consensus on the *positive* class, which again traces back to the absence of human ground truth. We use the lower-FP **llama-3.3-70b** as the default judge, and treat the escalated positives as candidates for human review rather than final labels.

Toward a continuous judge (proposed). All judge verdicts above are *discrete*: every escalated record resolves to a single leak/safe/uncertain label, and that is the mode used throughout this evaluation. We propose, but do not yet validate, a continuous extension based on *self-consistency*: query the judge k times at non-zero temperature and take the fraction of “leak” votes as a continuous leakage score in $[0, 1]$, feeding it directly into the aggregate PLR instead of the $\{0, 0.5, 1\}$ mapping. Genuinely ambiguous responses yield intermediate scores (split votes), while clear cases saturate, so the dispersion of an ensemble of judgments becomes the signal. We favour this over reading the provider’s token log-probabilities for the verdict token: a single token’s log-probability reflects the model’s confidence in *emitting that word*, not a calibrated probability that the response leaked, and post-RLHF models are systematically over-confident, so the resulting scores tend to collapse to 0 or 1 rather than spread. Self-consistency is also provider-agnostic (it needs only repeated sampling, not log-probability access, which several inference APIs do not expose), with log-probabilities as an optional cheaper fallback and the current discrete verdict as the final fallback. Validating whether the continuous score is better calibrated than the discrete one again requires ground truth; the verbatim-credential slice (Section 7.7) offers a small objective anchor on which to test it.

8 Discussion

Why ROUGE and NLI fail. The failure of lexical and entailment-based approaches is not a calibration artifact; it is structural. Modern instruction-tuned LLMs rephrase rather than regurgitate, collapsing lexical overlap to near-zero even for unambiguous leaks. NLI compounds this with 512-token truncation that silently discards the system prompt content being evaluated. Both methods are disqualified regardless of threshold tuning, and we report this as a useful negative result for practitioners evaluating off-the-shelf similarity metrics for leak detection.

The reranker’s conservatism: a detection asset and an operational cost. The reranker concentrates scores near 0 and 1 (mean MaxChunkSim 0.970 on Gemma adversarial records), so

when it misses a leak it tends to escalate rather than close: no confident wrong calls on Gemma, though the property weakens on more resistant agents (a few hard false negatives, Section 7.5), which is why the judge is not optional. The same conservatism is a cost on legitimate traffic: at $\tau = 0.6$ it closes only 4 of 16 Gemma legitimate records as *safe* versus cosine’s 10, escalating the rest at the full price of a judge call. The reranker-only pipeline is thus more expensive at scale than its detection numbers suggest. This is the central operational trade-off of the metric.

The domain-boundary false positive. False positives trace to an edge case inherent to semantic detection: agents answering legitimate queries in their own domain echo prompt language. On Gemma this is a single stable record across $\tau \in [0.55, 0.65]$, and the four-model false-positive rate stays in 3.6–8% at $\tau = 0.6$. One mitigation is per-query reference calibration, constructing G to reference the domain without sensitive specifics, sharpening Signal 2 on boundary cases.

Limitations.

- **Threshold calibration:** we adopt $\tau = 0.6$ as the cross-model default. Different system prompt styles, agent models, or domain vocabularies may shift the optimal threshold, so per-deployment calibration against a held-out legitimate sample is recommended before raising τ above this default.
- **No human ground truth:** this is the deepest limitation. Both the reference responses G and the leak/safe verdicts are produced by models: there is no human-labeled set establishing which responses *actually* leak. PLR is therefore an internally consistent operational score that we show generalizes across models and reference generators, not a detector validated against human judgment. A small human-labeled validation study is the most important next step before treating PLR as a ground-truth leak measure.
- **Scale:** the benchmark spans 246 queries answered by all four agent models (984 pooled records, Section 7.5). Aggregate confidence intervals are tight, but per-category and per-difficulty intervals remain wide.
- **Model-generated references:** every G is LLM-generated (Gemma for the origin queries, `gpt-oss-120b` for the new ones). We mitigate the risks with a hard anti-secret guard, human review, and cross-generator agreement (the two generators yield detection within two points for the Llama agents), but no G is human-verified.
- **Adversarial authorship skew:** `gpt-oss-120b` declined to author most extraction attacks for safety reasons, so the generated adversarial queries are predominantly authored by `llama-3.3-70b-versatile`. Attack-author diversity is thus lower than intended.
- **Label noise:** roughly 3% of generated adversarial queries are benign questions mislabeled as attacks. We retain them and rely on the origin subset to corroborate the conclusions; a label-cleaning pass would tighten the generated-subset estimates.
- **Multi-turn attacks:** the current pipeline evaluates single-turn responses. Multi-turn attacks, where an adversary extracts the prompt incrementally across several turns, are outside the current scope and are left for future work.
- **Undetected leaks:** a substantial share of adversarial records is not automatically classified by the reranker at $\tau = 0.6$ and is escalated to the judge, and, on the more resistant agents, a minority is closed as a hard false negative. These are medium-to-hard paraphrase attacks where the reranker score falls below threshold. A judge component is a required part of any production deployment.

Future work. The most important next step is a small human-labeled validation study to close the ground-truth gap; beyond it, a multi-turn evaluation protocol and a judge tuned for the ambiguous case are the primary extensions. A *continuous scoring mode* ($s_i = \text{MaxChunkSim}(R_i, P) \times (1 - \text{CosineSim}(R_i, G))$) is natural but currently degenerate: with the reranker, Signal 1 saturates near 1 for both leaks and legitimate domain references, collapsing the product to $1 - \text{CosineSim}(R_i, G)$. The deeper open problem is a deterministic *safe classifier*, one that confirms a response does not leak without an LLM judge and without cosine’s hard false negatives, which would let the pipeline close clean traffic cheaply and reserve the judge for genuinely ambiguous cases.

9 Conclusion

We introduce PLR (Prompt Leakage Resistance), a metric for quantifying an LLM agent’s resistance to system prompt extraction attacks. The metric combines a chunk-level maximum similarity score with a cosine alignment score against a safe reference response, applying a four-way decision table that separates confirmed leaks, safe responses, and judge-escalated ambiguous cases.

Our method comparison shows that lexical (ROUGE-L) and entailment-based (NLI) approaches are structurally incompatible with the task due to LLM paraphrase and token-limit truncation respectively, leaving the cross-encoder reranker as the strongest detector.

Extending the benchmark to 246 queries evaluated across four agent models, PLR produces a discriminative resistance ranking and its operating threshold $\tau = 0.6$ transfers across models, keeping false positives at or below 8%; a higher τ that looked optimal on a single model inflates false positives elsewhere. An independent labeled dataset corroborates the leak-overlap signal ($\text{AUC} \approx 0.99$) on the leak-versus-unrelated case, while the leak-versus-legitimate-on-topic case remains to be validated against human-labeled ground truth. We also report a smart cascade as a negative result: pre-filtering with cosine lowers judge load but converts ambiguous adversarial cases into hard false negatives, so it is not recommended as the primary configuration.

PLR is integrated into the Gaussia evaluation framework, enabling practitioners to compare model configurations on a common numeric scale across leakage categories and difficulty levels.

Open Source

PLR is developed as part of the Gaussia evaluation framework, an open-source initiative for reproducible LLM agent evaluation. The benchmark, experiment notebooks, and full implementation are publicly available as part of `pygaussia` at <https://github.com/gaussia-labs/pygaussia>.

References

- Pengcheng He, Xiaodong Liu, Jianfeng Gao, and Weizhu Chen. DeBERTa: Decoding-enhanced BERT with disentangled attention. In *International Conference on Learning Representations*, 2021.
- Bo Hui, Haolin Yuan, Neil Gong, Philippe Burlina, and Yinzhi Cao. PLeak: Prompt leaking attacks against large language model applications. In *Proceedings of the 2024 ACM SIGSAC Conference on Computer and Communications Security*. ACM, 2024. arXiv:2405.06823.
- Chin-Yew Lin. ROUGE: A package for automatic evaluation of summaries. In *Text Summarization Branches Out: Proceedings of the ACL-04 Workshop*. Association for Computational Linguistics, 2004.

- Rodrigo Nogueira and Kyunghyun Cho. Passage re-ranking with BERT. *arXiv preprint arXiv:1901.04085*, 2019.
- Fábio Perez and Ian Ribeiro. Ignore previous prompt: Attack techniques for language models. In *NeurIPS ML Safety Workshop*, 2022. arXiv:2211.09527. Best Paper Award.
- Qwen Team. Qwen3 technical report. Technical report, Alibaba Group, May 2025. arXiv:2505.09388.
- Nils Reimers and Iryna Gurevych. Sentence-BERT: Sentence embeddings using siamese BERT-networks. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing*. Association for Computational Linguistics, 2019.